UNIVERSITÀ DEGLI STUDI DI PERUGIA

Dipartimento di Matematica e Informatica

Progetto Introduzione Intelligenza Artificiale

Uniform Coloring

Autore: Hlade Carlos Floriano (367587)

A.A. 2025/2026

Indice

1 Descrizione formale del dominio e dei vincoli **3**
1.1 Descrizione formale del dominio 3
1.2 Descrizione formale dei vincoli 3

2 Struttura del programma **3**

3 Rete Neurale (Computer Vision) **4**
3.1 Preparazione del Dataset 4
3.2 Architettura del Modello 4

4 Image processing e costruzione degli stati **6**
4.1 Estrazione della griglia mediante Morfologia Matematica 6
4.2 Ritaglio intelligente e normalizzazione (Auto-Crop) 6

5 Algoritmi di ricerca e Solutore del Problema **7**
5.1 Modellazione del dominio (Problem) 7
5.2 Euristica (h_combined_cost) 8
5.3 Algoritmi utilizzati 8

6 Esecuzione e Statistiche dei Risultati **8**
6.1 Risultati Rete Neurale 8
6.2 Prestazioni degli Algoritmi di Ricerca 9

1 Descrizione formale del dominio e dei vincoli

1.1 Descrizione formale del dominio

Uniform Coloring è un dominio di pianificazione e ricerca in cui si ha a disposizione una griglia rettangolare composta da celle da colorare e una "testina" colorante (indicata con 'T'). Ogni cella, ad eccezione di quella iniziale della testina, possiede un colore di partenza fra tre opzioni disponibili: Blu (B), Yellow (Y) e Green (G). L'agente (la testina) può muoversi all'interno della griglia nelle quattro direzioni cardinali (Nord, Sud, Est, Ovest) con un costo uniforme pari a 1 per ogni spostamento. La testina ha inoltre la capacità di cambiare il colore della cella in cui si trova, applicando uno dei tre colori disponibili.

L'obiettivo finale (goal) è raggiungere una configurazione in cui tutte le celle della griglia abbiano lo stesso colore (non importa quale esso sia) e la testina sia tornata esattamente nella sua posizione di partenza. I costi delle azioni di colorazione sono differenziati: $cost(B) = 1$, $cost(Y) = 2$, $cost(G) = 3$.

1.2 Descrizione formale dei vincoli

Le azioni eseguibili dall'agente e le regole del dominio devono rispettare i seguenti vincoli:

- L'agente si può muovere solo fra celle adiacenti (Nord, Sud, Est, Ovest) e non in diagonale.
- L'agente non può compiere movimenti che lo portino al di fuori dei confini fisici della griglia rettangolare.
- La testina colora la cella su cui si trova in quel momento. Durante lo spostamento, la cella abbandonata assume il colore appena applicato.
- La cella di partenza della testina (Start Position) non ha un colore proprio iniziale. Per definizione del dominio implementato, la testina non colora mai la propria casella di partenza.
- Il costo delle azioni si accumula ad ogni passo: gli spostamenti costano sempre 1, mentre i costi di colorazione dipendono dal pigmento scelto.
- L'agente può compiere solo un passo alla volta.

2 Struttura del programma

Il progetto è stato sviluppato in Python, separando logicamente la fase di classificazione visiva dalla ricerca della soluzione. I file principali sono:

- `requirements.txt`: file necessario per l'installazione delle librerie necessarie (TensorFlow, OpenCV, Numpy, AIMA, ecc.).
- `keras_CNN.py`: script dedicato al caricamento del dataset EMNIST, al pre-processing dei dati, alla costruzione, addestramento e salvataggio della Rete Neurale Convoluzionale (CNN).
- `UniformColoring.py`: è il modulo principale che unisce la computer vision per l'estrazione della griglia, la definizione del problema (come sottoclasse `Problem` della libreria AIMA) e l'esecuzione degli algoritmi di ricerca.

3 Rete Neurale (Computer Vision)

Per riconoscere il contenuto delle celle (le lettere B, G, T, Y) è stata addestrata una Rete Neurale Convoluzionale (CNN) sul dataset EMNIST-letters.. Poiché il dominio richiede di riconoscere lettere scritte a mano all'interno delle celle della griglia, una CNN risulta l'architettura ideale grazie alla sua capacità di estrarre e combinare pattern visivi (feature) in modo gerarchico.

3.1 Preparazione del Dataset

Il dataset originale contiene l'intero alfabeto, ma per il nostro dominio ci interessano solo quattro classi specifiche. I dati sono stati perciò sottoposti a un rigoroso pre-processing:

- **Filtraggio e Rimappatura:** Sono state estratte solo le immagini corrispondenti agli indici di B (2), G (7), T (20) e Y (25). Le etichette sono state rimappate in un nuovo range $[0, 3]$ per adattarsi al classificatore finale.
- **Trasposizione e Reshape:** Le immagini di EMNIST si presentano ruotate e specchiate nel file `.mat`. È stata perciò applicata una trasposizione degli assi per riportare le lettere al corretto orientamento visivo, garantendo una forma finale di $28 \times 28 \times 1$ (immagini in scala di grigi a singolo canale).
- **Normalizzazione e Codifica:** I valori dei pixel sono stati normalizzati scalandoli dal range $[0, 255]$ all'intervallo continuo $[0, 1]$, facilitando la convergenza dei pesi durante la backpropagation. Le etichette sono state codificate tramite *One-Hot Encoding*.

3.2 Architettura del Modello

L'architettura costruita è di tipo sequenziale. È stata progettata su misura per estrarre feature robuste da caratteri manoscritti, gestendone le inevitabili irregolarità. Di seguito è riportata l'analisi dettagliata del flusso dei dati all'interno della rete:

- **Layer di Input e Data Augmentation:** La rete riceve in input tensori di dimensione $28 \times 28 \times 1$. Per rendere il modello invariante alle piccole imprecisioni della scrittura umana, i primi layer applicano trasformazioni casuali (attive solo durante la fase di training):
  - `RandomRotation(0.06)`: Applica una rotazione di circa $\pm 22°$, simulando la naturale inclinazione della grafia.
  - `RandomZoom(-0.15, 0.05)`: Applica uno zoom asimmetrico. È più probabile che una lettera venga scritta piccola (zoom out fino al 15%) piuttosto che sbordare fuori dalla cella (zoom in massimo del 5%).
  - `RandomTranslation(0.08, 0.08)`: Trasla l'immagine di circa ± 2 pixel per simulare una centratura imperfetta.

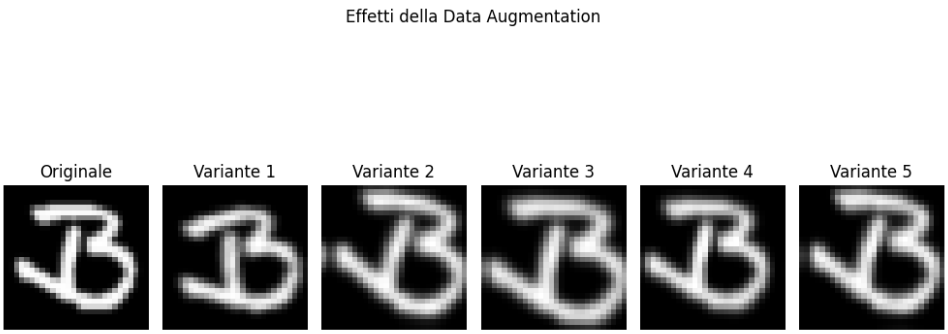

Figura 1: Data Augmentation applicata alla lettera: B

- **Primo Blocco Convoluzionale (Estrazione di basso livello):**
  - **Conv2D (32 filtri 3×3):** I filtri scorrono sull'immagine per individuare feature semplici, come bordi, angoli e linee. L'uso del `padding='same'` mantiene inalterata la dimensione spaziale (28×28). Viene usata la funzione di attivazione **ReLU** (Rectified Linear Unit) che azzera i valori negativi, introducendo non-linearità.
  - **MaxPooling2D (2×2):** Riduce la dimensionalità tenendo solo il valore massimo in ogni finestra 2×2. Questo passaggio comprime la feature map a 14×14, riducendo il carico computazionale ed estraendo i pattern più forti.
- **Secondo Blocco Convoluzionale (Estrazione di medio livello):**
  - **Conv2D (64 filtri 3×3):** Aumentando i filtri, la rete impara a combinare i bordi trovati dal livello precedente in forme più complesse (es. curve chiuse, intersezioni di tratti). L'output resta 14×14.
  - **MaxPooling2D (2×2):** Dimezza nuovamente la risoluzione spaziale, portando la mappa a 7×7.
- **Terzo Blocco Convoluzionale (Rappresentazione ad alto livello):**
  - **Conv2D (128 filtri 3×3):** In questa fase, le forme vengono combinate in rappresentazioni di alto livello (la struttura logica della lettera). Il numero di filtri è stato raddoppiato rispetto al blocco precedente per garantire un'elevata capacità espressiva.
  - *Nota progettuale:* Qui **non** è stato inserito un ulteriore layer di Pooling. Essendo la dimensione arrivata a soli 7×7 pixel, un'ulteriore riduzione spaziale avrebbe distrutto preziose informazioni strutturali prima della classificazione.
- **Classificatore Finale (Fully Connected):**
  - **Flatten:** Appiattisce le feature map tridimensionali ($7 \times 7 \times 128$) in un unico array monodimensionale di 6272 elementi. La rete ora smette di guardare la "posizione" spaziale dei tratti e si concentra solo sulla loro "presenza".
  - **Dense (128 neuroni, ReLU):** Ogni neurone è collegato a tutti i 6272 input. Funge da "riassunto" delle feature estratte, imparando le correlazioni logiche per distinguere le lettere.

- **Dropout (0.4):** Durante l'addestramento, il 40% dei neuroni di questo strato viene "spento" casualmente. Questo obbliga la rete a non affidarsi a singoli percorsi mnemonici, prevenendo in modo efficace l'overfitting.
- **Dense di Output (4 neuroni, Softmax):** L'ultimo strato possiede un neurone per ogni classe (B, G, T, Y). La funzione di attivazione **Softmax** converte i valori in una distribuzione di probabilità la cui somma è 1. Il neurone con la probabilità maggiore decreta la predizione finale per quella cella.

La rete viene infine compilata utilizzando la funzione di costo `categorical_crossentropy` (standard per problemi di classificazione multiclasse) e l'algoritmo di ottimizzazione **Adam**, noto per la sua convergenza rapida e stabile, specialmente in presenza di pesanti layer di Data Augmentation.

4 Image processing e costruzione degli stati

La seconda fase del progetto prevede l'estrazione delle informazioni strutturali e visive a partire dall'immagine della griglia fornita in input. Per raggiungere questo obiettivo, l'intero modulo di visione artificiale è stato progettato e sviluppato sfruttando le potenzialità della libreria OpenCV. Lo scopo è duplice: isolare le singole celle che compongono la griglia e preparare il contenuto di ciascuna cella per renderlo compatibile con il formato del dataset EMNIST richiesto dalla rete neurale.

4.1 Estrazione della griglia mediante Morfologia Matematica

Il primo ostacolo affrontato è stato il riconoscimento automatico della struttura a griglia, a prescindere dallo spessore delle linee disegnate. Il processo implementato segue questi step:

- **Binarizzazione iniziale:** L'immagine viene convertita in scala di grigi e sottoposta a un *threshold* inverso a soglia fissa (valore 200 su 255). In questo modo, le linee scure della griglia diventano pixel bianchi su uno sfondo completamente nero, facilitando l'analisi spaziale.
- **Rilevamento morfologico:** Per isolare le linee orizzontali e verticali sono state applicate operazioni di morfologia matematica, in particolare l'apertura (`MORPH_OPEN`). Sono stati creati due kernel rettangolari dinamici, proporzionali alle dimensioni dell'immagine (1/10 della larghezza per le orizzontali e 1/10 dell'altezza per le verticali). Questo passaggio isola l'ossatura della griglia filtrando gli elementi interni.
- **Intersezione logica:** Eseguendo un'operazione di `bitwise_and` (AND logico) tra l'immagine delle linee verticali e quella delle orizzontali, si ottengono unicamente i punti di intersezione, ovvero i vertici fisici delle celle.
- **Clustering delle coordinate:** Poiché una linea spessa genera multipli pixel di intersezione vicini tra loro, è stata implementata la funzione `cluster_positions`: essa ordina i pixel trovati e raggruppa quelli distanti meno di una certa soglia (es. 10 pixel) l'uno dall'altro. Il risultato è un elenco pulito ed esatto delle coordinate X e Y che delimitano le righe e le colonne.

4.2 Ritaglio intelligente e normalizzazione (Auto-Crop)

Una volta note le coordinate, l'immagine originale viene divisa in singole celle (Region of Interest o ROI). Tuttavia, passare la ROI grezza alla rete neurale porterebbe a predizioni errate, poiché le lettere potrebbero essere decentrate o di dimensioni non standard. Per risolvere questo problema, la funzione `auto_crop_cell` elabora le celle nel seguente modo:

1. **Metodo di Otsu Inverso:** La cella viene binarizzata usando il metodo di Otsu (`THRESH_OTSU`), che calcola automaticamente la soglia ottimale in base all'istogramma dei grigi della specifica cella, facendo risaltare il tratto della lettera su sfondo nero.

2. **Bounding Box del contorno maggiore:** Tramite `findContours` (`RETR_EXTERNAL`) si individuano tutte le forme nella cella. Viene selezionato solo il contorno con l'area maggiore (per ignorare l'eventuale rumore) e se ne estrae il rettangolo di delimitazione minimo (*Bounding Box*).

3. **Canvas quadrato e Padding:** Viene creato un "canvas" nero quadrato. Il lato di questo canvas è pari al lato maggiore del bounding box maggiorato di un *padding* del 20%. La lettera estratta viene incollata esattamente al centro del canvas. Questo artificio è cruciale per replicare il formato in cui le lettere EMNIST sono centrate e spaziate dai bordi.

4. **Ridimensionamento EMNIST:** L'immagine perfettamente centrata e quadrata viene infine ridimensionata a 28×28 pixel usando un'interpolazione ad area (`INTER_AREA`), ottima per la riduzione delle dimensioni.

Le celle così elaborate vengono raccolte in un unico *batch* di array Numpy, normalizzate dividendole per 255.0 e passate alla CNN in una singola chiamata predittiva, ottimizzando drasticamente i tempi di esecuzione. A valle della predizione, un blocco logico verifica la corretta rilevazione della testina "T"; in assenza di quest'ultima, l'esecuzione viene interrotta notificando l'anomalia, garantendo la robustezza dell'input prima di avviare l'algoritmo di ricerca.

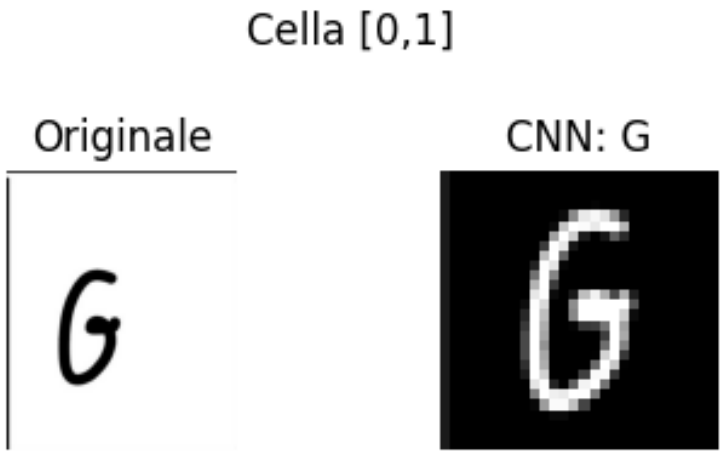

Figura 2: Image Processing

5 Algoritmi di ricerca e Solutore del Problema

5.1 Modellazione del dominio (Problem)

Il problema è stato modellato implementando la classe `UniformColoring`, derivata dalla classe base `Problem` della libreria AIMA. Lo stato è rappresentato dalla classe `ColoringState`, caratterizzata da:

- `grid`: Una tupla che rappresenta i colori di tutte le celle (immutabile e quindi *hashable*).
- `held_color`: Il colore della cella attualmente nascosto sotto la testina 'T'.
- `t_pos`: L'indice precalcolato della posizione corrente della testina.

I metodi principali sono stati così ridefiniti:

- `actions`: Genera le azioni valide (N, S, W, E) verificando i bordi della griglia, e le azioni di colore (COL-B, COL-Y, COL-G) solo se non ci si trova sulla *start position*.
- `result`: Dato uno stato e un'azione, ritorna lo stato successivo aggiornando la posizione, il colore della cella liberata e il nuovo `held_color`.
- `path_cost`: Associa alle azioni i costi differenziati previsti dal dominio.
- `goal_test`: Verifica che la testina si trovi nella posizione di partenza e che tutte le celle (escludendo 'T' e 'None') abbiano esattamente lo stesso colore.

5.2 Euristica (h_combined_cost)

È stata definita un'euristica `h_combined_cost` per ottimizzare la ricerca:

$$h(n) = h_{dist}(n) + h_{paint}(n)$$

- h_{dist}: Distanza di Manhattan dalla posizione attuale alla casella di partenza. L'agente *deve* percorrerla per raggiungere il goal.
- h_{paint}: Il costo teorico minimo per uniformare le celle attualmente visibili. Viene calcolato come il minimo tra i costi per dipingere le celle rimanenti di B, Y o G.

Ammissibilità e consistenza: L'euristica è **ammissibile** in quanto sottostima sempre il costo reale: somma la distanza fisica e i costi puri di colore ignorando i movimenti necessari per spostarsi da una cella all'altra durante la colorazione.
Analisi delle varianti: Se il costo di spostamento costasse 0, l'euristica perderebbe l'ammissibilità perché h_{dist} aggiungerebbe un costo non reale. Se invece i costi di colorazione fossero uniformi, h_{paint} rimarrebbe valida semplicemente moltiplicando il numero di celle "sbagliate" per il costo base, rimanendo una stima per difetto.

5.3 Algoritmi utilizzati

Sono stati eseguiti e confrontati tre algoritmi di ricerca su grafo:

- **Uniform Cost Search (UCS)**: Ricerca non informata, garantisce ottimalità.
- **Greedy Best First Search**: Ricerca informata, si guida solo sull'euristica, rapida ma non garantisce il costo minimo.
- **A* Search**: Ricerca informata ($f(n) = g(n) + h(n)$), sfrutta l'euristica garantendo la soluzione ottima in presenza di euristiche ammissibili.

6 Esecuzione e Statistiche dei Risultati

6.1 Risultati Rete Neurale

Durante il training (8 epoche), il modello ha mostrato i seguenti risultati:

- **Test Accuracy:** %
- **Test Loss:**

Figura 3: Grafici di Model Accuracy e Model Loss.

Il modello si è rivelato estremamente robusto nel classificare le celle reali generate dall'algoritmo di visione artificiale, grazie anche alla data augmentation applicata in fase di addestramento.

6.2 Prestazioni degli Algoritmi di Ricerca

Testando il sistema su diverse configurazioni di immagini (es. griglie di varie dimensioni passate in input), si possono osservare i tempi di convergenza e i nodi esplorati.

Algoritmo	**Nodi generati**	**Percorsi esplorati**	**Costo Path**	**Tempo (s)**
UCS	*dato*	*dato*	*dato*	*dato*
Greedy	*dato*	*dato*	*dato*	*dato*
A*	*dato*	*dato*	*dato*	*dato*

Tabella 1: Confronto prestazioni risoluzione di un problema medio.

(Aggiungere qui un commento sui risultati reali che otterrai: tipicamente UCS esplora moltissimi nodi, Greedy è rapidissimo ma potrebbe sbagliare il costo ottimale, mentre A offre il perfetto bilanciamento riducendo drasticamente i nodi di UCS e mantenendo il costo ottimo del percorso.)*

Figura 4: Output a terminale della soluzione trovata.